Plus 宽范围混合近似工具集技术说明

1. 技术方案原理

plus_hybrid_approx 对应增强混合近似协议，用于宽范围输入上的 Sigmoid+、Tanh+、Erf+、Exp+、Inv+ 和 InvSqrt+。与 base 版本不同，plus 版本不假设输入天然落在单一拟合区间内，而是在 base approximation 外增加比较、分段、饱和和选择逻辑。

bounded 函数使用区间门控：区间外直接输出饱和值，区间内调用 base 混合近似。unbounded 函数使用分段近似：不同输入区间选择不同参数或不同缩放策略，再用 Newton 迭代提升精度。比较门由 ODReLU/CMP 类协议生成，门控选择由 SMP/3SMP 完成。

```
宽范围输入
  -> comparison gates
  -> base candidates / segment candidates
  -> secure select
  -> 可选 Newton 修正
  -> 输出分享
```

我们将增强协议记为 Π_θ^+。它不是替代 base，而是在 base 的局部近似能力之外增加 wide-range support。bounded 函数的增强来自 GeLU/SiLU 类三段思想；unbounded 函数的增强来自 LayerNorm 中 InvSqrt 宽范围处理思想。

1.1 模块定位

plus_hybrid_approx 位于 base_hybrid_approx 之上，主要解决 base 协议输入范围受限的问题。base 版本假设输入已经落在某个拟合区间内；plus 版本则把“区间判断、候选值计算、安全选择、必要时迭代修正”组合成完整协议，使上层 Softmax、GeLU、SiLU、LayerNorm 等算子可以直接处理更宽的真实输入分布。

从功能上看，plus 协议不是单纯提高某个局部函数的拟合阶数，而是改变宽范围处理方式：对 bounded 函数使用左右饱和；对 Exp 使用 clipping 或分段参数；对 Inv/InvSqrt 使用 steep/flat 初值再进入 Newton refinement。因此 plus 的开销通常高于 base，但它避免了把一个低阶近似强行套到过宽区间上。

函数	base 问题	plus 增强方式	典型上层用途
Sigmoid+	区间外趋近 0/1，base 尾部误差增大	左右饱和 + 中间 base	SiLU、门控
Tanh+	宽区间边界振荡或饱和误差	左右饱和 + 中间 base	激活函数
Erf+	GeLU 尾部误差会被输入幅值放大	左右饱和 + 中间 base	GeLU

函数	base 问题	plus 增强方式	典型上层用途
Exp+	指数增长快，单区间拟合不稳定	clipping / 分段 Exp base	Softmax
Inv+	靠近 0 区域变化剧烈	steep/flat 初值 + Newton	Softmax 分母倒数
InvSqrt+	小方差区域敏感，宽范围初值难拟合	steep/flat 初值 + Newton	LayerNorm

1.2 系统参与方与安全边界

plus 协议沿用二方加性分享和 TEE 辅助结构。CSP0/CSP1 持有输入分享和输出分享，REE/App 负责协议调度、网络收发和 ECALL 调用，TEE/Enclave 提供比较、掩码和候选计算所需的辅助材料。

与 base 相比，plus 的敏感信息更多：不仅有输入值 x，还有区间归属信息，例如 `x < lower`、`x > upper`、x 属于 steep 还是 flat。plus 协议的安全边界要求这些 gate 也保持为分享，不能在 REE 或网络中以明文形式暴露。也就是说，协议可以在分享域里使用 gate 做选择，但不能让任一方直接知道某个样本落在哪个区间。

1.3 plus 与 base 的组合关系

plus 协议通常包含三类子模块。第一类是比较 gate，用来判断输入所在区间；第二类是候选值，包括常数饱和值、base approximation 候选值或不同 segment 的候选值；第三类是 secure select，用 gate 把候选值合成为最终输出分享。

这种组合关系可以写成统一形式：

```
output = select(gates, candidates)
```

其中 `gates` 和 `candidates` 都是分享值。bounded 函数的候选值通常是 `{left_constant, base(x), right_constant}`；Exp+ 的候选值是 clipped/segmented 后的指数近似；Inv+/InvSqrt+ 的候选值是不同区间的 Newton 初值。只要 gate 不公开、候选值不公开、选择过程用 SMP/3SMP 完成，最终输出仍是加性分享。

2. 逻辑推导

2.1 Bounded 函数三段选择

以 bounded 函数为例，设 base 拟合区间为 `[lower, upper]`。协议先生成 `left = [x < lower]`、`right = [x > upper]`，中间候选为 `mid = base(x)`。输出可写为 `y = c_left * left + mid * (1 - left - right) + c_right * right`，其中 Sigmoid 的 `c_left,c_right` 为 `0,1`，Tanh/Erf 的饱和值为 `-1,1`。因为 `left,right,mid` 都是分享值，`mid * (1-left-right)` 需要安全乘法或等价融合协议。

也可以写成安全 mux 形式 `select(left, c_left, select(right, c_right, base(x)))`。在分享域中，`select(g,a,b)` 通常实现为 `b + g * (a-b)`，因此每次选择都可归约为一次安全乘法。我们使用 3SMP+ 类融合协议减少组合开销。

三段选择的关键不是把左右区间算得更精确，而是利用 bounded 函数的数学性质：当输入足够远离中心区间时，真实函数已经接近常数。此时继续调用 base 近似既增加开销，也可能在边界外产生不必要的振荡误差。plus 版本用安全比较判断是否进入饱和区，再用安全选择把常数候选和 base 候选合并，避免在宽范围上强行拟合一个函数。

需要注意的是，`left` 和 `right` 应互斥。实际阈值必须满足 `lower < upper`，并且比较协议的边界定义要一致，例如左侧使用 `< lower`，右侧使用 `> upper`，中间自然覆盖闭区间或半开区间。边界点的选择会影响极少量样本的输出，但不会改变整体协议结构。

2.2 Exp+ 的区间裁剪与分段

Exp+ 先通过比较/裁剪把输入落到稳定区间，再在近区间和远区间选择不同拟合参数。其目标不是在无限区间拟合 e^x，而是把实际使用的输入压到一个有限且数值稳定的区间。抽象地说，协议先构造 `x_clip = clip(x, alpha, beta)`，再计算 `y = exp_base_or_segment(x_clip)`。对于远离主区间的输入，协议可以采用更适合该子区间的拟合参数，避免单一近似函数同时覆盖陡峭区域和平坦区域。

在 Softmax 场景中，Exp+ 的目标通常不是独立还原任意实数上的 `exp(x)`，而是为归一化概率提供稳定的正值近似。clipping 会引入可控偏差，但可以显著降低溢出风险和宽范围拟合难度。若上层已经做了减最大值或范围裁剪，Exp+ 的区间可以更窄；若上层没有可靠范围控制，Exp+ 就必须承担更多 clipping 和分段职责。

2.3 Inv+ 与 InvSqrt+ 的分段初值

Inv+ 和 InvSqrt+ 按输入大小分成 steep 与 flat 区域，通过 `select(x in steep, approx_steep(x), approx_flat(x))` 得到初值 y_0。之后使用 Newton 迭代：

```
Inv:      y <- y * (2 - x*y)
InvSqrt:  y <- y * (3 - x*y²) / 2
```

其中 `approx_steep` 和 `approx_flat` 都可以调用 base hybrid approximation，只是参数和拟合区间不同。分段的作用是让 Newton 迭代的初值足够接近真实值，从而用较少轮数达到目标精度。

steep/flat 的分界点需要同时考虑函数形状和定点数值范围。分界点过低时，flat 区间仍然包含变化很快的输入，初值误差会变大；分界点过高时，steep 区间过宽，局部拟合压力增加，并且更多样本会走高成本候选。plus 协议的实际调参目标是在最大误差、平均误差和通信/乘法开销之间取得平衡。

2.4 参数、gate 与误差传播

plus 协议的参数包括 base 近似参数、区间阈值、clipping 边界、steep/flat 分界点和 Newton 迭代轮数。它们共同决定输出精度，而不是某一组多项式或三角系数单独决定精度。调参时应先确认 base 候选在各自局部区间内误差可接受，再检查 gate 是否把真实输入分布正确分配到候选区间。

gate 误差或边界定义不一致会直接变成分支选择错误。对于 bounded 函数，这可能把本应饱和的输入送入 base 近似，或把中间区间输入提前截断为常数；对于 Inv/InvSqrt，这可能选择错误初值，使 Newton 迭代从较差初值开始。由于选择本身在分享域内完成，调试时通常需要在测试模式下重构统计信息，分别观察各区间样本数量和区间内误差。

plus 的误差传播也比 base 更复杂。base 误差主要来自局部拟合和定点缩放；plus 还包含 clipping 偏差、分段边界误差、secure select 乘法误差和 Newton 初值误差。评估时应同时报告平均误差、最大误差、输入范围、各分段样本占比和协议开销，否则单独的平均误差可能掩盖边界区间的最大偏差。

3. 详细理论解释

我们的核心观察是：Transformer 非线性函数的输入范围往往比固定小区间更宽。plus 版本的每个协议都在 base 近似外增加宽范围处理逻辑。下面按协议分别解释，重复的比较门、候选值和安全选择逻辑也分别写出。

3.0 共同安全与正确性模型

plus 协议的正确性可以拆成三个层次。第一，比较 gate 要正确表达输入所在区间；第二，每个候选值要在对应区间内近似目标函数或目标饱和值；第三，secure select 要按照 gate 把候选值组合成最终输出。只要这三层都成立，重构后的输出就接近对应的 wide-range 目标函数。

安全性同样来自组合。比较协议只输出分享 gate，不公开比较结果；base approximation 只输出候选分享，不公开中间明文；SMP/3SMP 只在分享域中执行候选合成。对于 adversary 来说，额外的 plus 逻辑不应泄露“输入落在哪个区间”这一侧信道信息。

3.1 Bounded 函数 plus 协议

Sigmoid+、Tanh+ 和 Erf+ 都利用函数两端饱和的性质，把宽范围输入拆成左饱和区、中间 base 近似区和右饱和区。三者共享“比较 gate + base 候选 + 安全选择”的结构。

这类协议的设计重点是阈值选择。阈值太窄会让较多输入直接进入饱和常数，造成中心区间外但尚未完全饱和位置的误差；阈值太宽则会让 base 近似承担过宽区间，增加振荡或边界误差。实际阈值应结合模型激活分布和函数尾部变化速度确定。

3.1.1 Sigmoid+ 协议

Sigmoid+ 用于宽范围 `sigmoid(x) = 1 / (1 + exp(-x))`。Sigmoid 在左端趋近 0，在右端趋近 1，因此 Sigmoid+ 使用三段逻辑：`x < lower` 时输出 0，`x > upper` 时输出 1，其余情况输出 `sigmoid_base(x)`。

在分享域中，协议不能直接执行明文分支，因此先用 ODReLU/CMP 生成两个 gate：`g_left=[x<lower]` 和 `g_right=[x>upper]`，再令 `g_mid = 1 - g_left - g_right`。最终输出分享可写为 `0*<g_left> + <sigmoid_base(x)>*<g_mid> + 1*<g_right>`。Sigmoid+ 的正确性来自三段函数定义：左侧饱和为 0，右侧饱和为 1，中间由 base 近似。安全性来自比较门和 base 近似都是分享输出，安全选择只调用 SMP/3SMP，不公开输入所在区间。

Sigmoid+ 对 SiLU 或门控结构尤其重要。若尾部仍使用 base 近似，左侧小概率区域可能产生非零偏差，右侧接近 1 的区域也可能出现轻微过冲。三段饱和可以把这些尾部误差压到常数选择问题上，使主要近似压力集中在函数斜率最大的中心区间。

3.1.2 Tanh+ 协议

Tanh+ 用于宽范围 `tanh(x) = (exp(x)-exp(-x))/(exp(x)+exp(-x))`。Tanh 在左端趋近 -1，在右端趋近 1，因此 Tanh+ 的三段逻辑是：`x < lower` 时输出 -1，`x > upper` 时输出 1，其余情况输出 `tanh_base(x)`。协议同样先生成 `g_left` 和 `g_right`，再得到 `g_mid`，输出分享可写为 `(-1)*<g_left> + <tanh_base(x)>*<g_mid> + 1*<g_right>`。Tanh+ 的正确性依赖 Tanh 的饱和性质和 base 区间近似精度。安全性上，比较结果不以明文暴露，`tanh_base(x)` 也是分享值，最终选择过程只产生输出分享。即使某一方参与了比较和选择，也只能看到被掩码保护的消息。

Tanh+ 的左右饱和值对称，因此它比 Sigmoid+ 更依赖边界处的连续性。如果 `tanh_base(lower)` 与 -1 或 `tanh_base(upper)` 与 1 差距较大，分段点附近会出现可见跳变。实际参数应让 base 区间边界已经足够接近饱和值，或把阈值设置在误差可接受的位置。

3.1.3 Erf+ 协议

Erf+ 用于宽范围误差函数 `erf(x) = 2/sqrt(pi) * integral_0^x exp(-t²) dt`。Erf 与 Tanh 类似，两端分别趋近 -1 和 1，因此 Erf+ 的逻辑是：`x < lower` 时输出 -1，`x > upper` 时输出 1，其余情况输出 `erf_base(x)`。协议通过两个安全比较得到左右区间 gate，通过 SMP/3SMP 将 `erf_base(x)` 和两个常数饱和值组合为最终输出。Erf+ 的正确性很重要，因为 GeLU 会调用 Erf/Erf+；若 Erf+ 在尾部不做饱和而只使用 base 拟合，尾部误差会被 GeLU 中的乘法项放大。Erf+ 的安全性与 Sigmoid+/Tanh+ 相同：区间判断是分享 gate，候选值是分享，选择过程不公开分支。

Erf+ 的误差会进入 GeLU 的乘法项 `0.5*x*(1+erf(x/sqrt(2)))`。这意味着同样大小的 Erf 误差，在大幅值输入上会造成更大的 GeLU 输出误差。因此 Erf+ 的边界处理比普通 bounded 激活更关键：尾部应尽快进入饱和常数，中心区间再由 base 提供平滑近似。

3.2 Exp+ 宽范围指数协议

Exp+ 不能直接使用 bounded 函数的常数饱和策略。它通过裁剪和分段近似把指数增长控制在稳定区间内，主要服务于 Softmax 等需要大量指数计算的上层协议。

3.2.1 Exp+ 协议

Exp+ 用于宽范围 `exp(x)`。指数函数增长很快，不能像 bounded 函数那样简单用两端常数饱和。Exp+ 的核心是裁剪和分段近似：先构造 `x_clip = clip(x, alpha, beta)`，再计算 `y = exp_base_or_segment(x_clip)`。协议先用安全比较得到 x<alpha 和 x>beta 的 gate，再构造 clipped 输入分享。随后在稳定区间内调用 Exp base，或根据区间选择不同 Exp base 参数。Exp+ 的正确性是近似正确性：它近似的是裁剪后的指数值，并通过合理选择 `alpha,beta` 保持目标任务精度。Softmax 中使用 Exp+ 时，clipping 可以避免安全最大值搜索，并控制指数爆炸。

Exp+ 的安全性来自两部分：clipping 的比较 gate 不公开，Exp base 的输入和输出都是分享。协议不会公开 x 是否被裁剪，也不会公开 `exp(x_clip)` 明文。

Exp+ 的阈值通常与 Softmax 的数值策略绑定。若输入已经减去行最大值，则理论上最大值附近为 0，负值区间决定了概率尾部。此时 `alpha` 控制尾部概率是否被截断，`beta` 通常控制上界稳定性。选择过大的区间会增加 Exp base 拟合难度；选择过小的区间会把小概率项过早裁剪，影响概率分布尾部。

3.3 分段迭代型 plus 协议

Inv+ 和 InvSqrt+ 的共同点是先通过分段得到足够好的初值，再用 Newton refinement 收敛。分段 gate、候选初值和迭代中间值都保持为分享。

3.3.1 Inv+ 协议

Inv+ 用于宽范围倒数 `inv(x) = 1 / x`。倒数在靠近 0 时变化剧烈，在较大输入区域变化平缓。Inv+ 因此采用 steep/flat 分段：steep 区域使用 `inv_base_steep(x)` 作为初值，否则使用 `inv_base_flat(x)`。分段选择后，协议执行 Newton refinement `y <- y * (2 - x*y)`。Inv+ 的正确性来自两个层次。第一，steep/flat 分段保证初值 y_0 在各自区间内足够接近 `1/x`；第二，Newton 迭代把初值误差进一步压低。安全性上，分段 gate 是分享，两个候选初值是分享，Newton 中的 `x*y` 和 `y*(2-x*y)` 都通过 SMP 完成，因此整个过程不会公开 x、区间或倒数结果。

Inv+ 在 Softmax 中通常作用于行和的倒数。行和始终为正，但会受序列长度、Exp+ clipping 和定点缩放影响。若只使用单一 base 初值，短行、小行和或长行、大行和可能落在同一拟合区间的不同极端，导致 Newton 初值质量不均衡。steep/flat 分段可以让小值区域和大值区域分别使用更合适的参数。

3.3.2 InvSqrt+ 协议

InvSqrt+ 用于宽范围倒平方根 `invsqrt(x) = 1 / sqrt(x)`。该协议主要服务于 LayerNorm，因为 `var+eps` 的范围可能很宽。InvSqrt+ 同样使用 steep/flat 分段：steep 区域使用 `invsqrt_base_steep(x)` 作为初值，否则使用 `invsqrt_base_flat(x)`。随后执行 Newton refinement：

```
y <- y * (3 - x*y²) / 2
```

InvSqrt+ 的正确性要求初值进入 Newton 收敛域。分段近似使小输入区域和大输入区域分别使用合适的拟合参数，从而避免单一 base 在宽区间上误差过大。安全性上，y^2、$x*y^2$ 和最终更新都在分享域内通过 SMP/3SMP 完成；比较 gate、候选值和迭代中间值均不公开。

InvSqrt+ 对 LayerNorm 的影响比单点误差更明显，因为同一个 `1/sqrt(var+eps)` 会乘到一整行归一化结果上。若倒平方根估计偏大，整行输出会被放大；若估计偏小，整行输出会被压缩。plus 分段的目的就是在小方差敏感区和大方差平缓区都提供稳定初值，减少 Newton 迭代前的系统偏差。

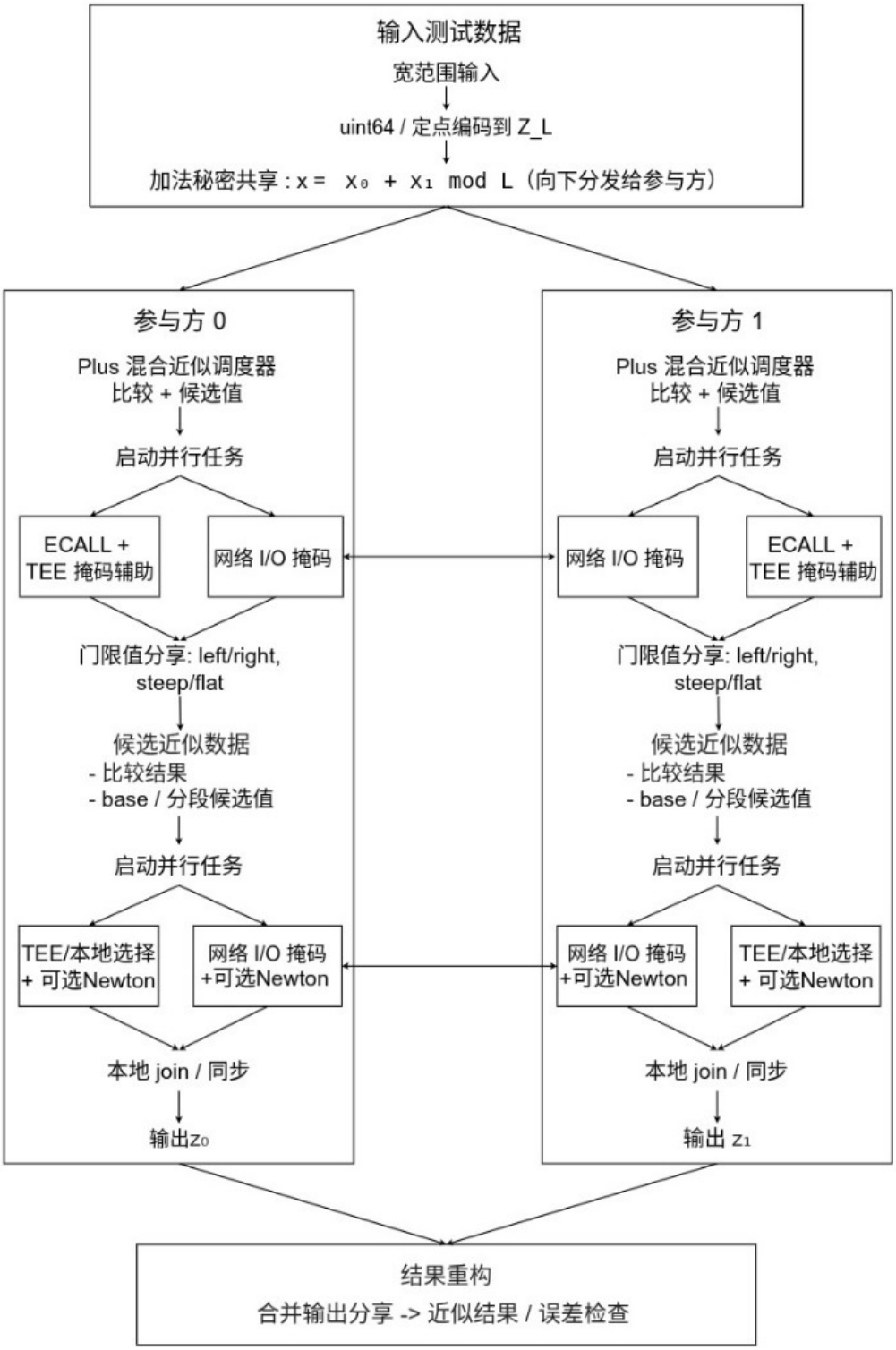
3.4 plus 协议的共同实现含义

每个 plus 协议都比对应 base 协议多出比较和选择开销。实现中需要把 ODReLU/CMP、base approximation、SMP/3SMP 组织为并行路径：TEE 负责掩码材料和辅助计算，Network I/O 负责交换掩码消息，最后本地同步得到输出分享。参数选择时需要同时看最大误差、平均误差和通信轮次；只扩大输入范围而不调整分段，会导致近似误差上升。

从工程角度看，plus 的主要成本来自三处：比较 gate、多个候选值计算、secure select。bounded 函数通常只有一个 base 候选和两个常数候选，额外成本集中在比较和选择；Inv+/InvSqrt+ 需要计算两个初值候选并执行 Newton refinement，乘法开销更高；Exp+ 的成本取决于是简单 clipping 还是多段参数选择。

因此评估 plus 协议时不能只看单次函数延迟，还要看输入分布。若大多数输入落在饱和区，bounded plus 可能比总是调用 base 更高效；若大多数输入落在中间区，plus 会比 base 多出比较和选择开销。对于 Inv+/InvSqrt+，如果分段显著减少 Newton 轮数或降低最大误差，额外候选计算就是合理的；否则应重新检查分界点和初值参数。

4. 算法框架图


输入测试数据
宽范围输入
uint64 / 定点编码到 Z_L
加法秘密共享 : x = x₀ + x₁ mod L（向下分发给参与方）
参与方 0
Plus 混合近似调度器
比较 + 候选值
启动并行任务
ECALL +
TEE 掩码辅助
网络 I/O 掩码
门限值分享: left/right,
steep/flat
候选近似数据
- 比较结果
- base / 分段候选值
启动并行任务
TEE/本地选择
+ 可选Newton
网络 I/O 掩码
+可选Newton
本地 join / 同步
输出z₀
参与方 1
Plus 混合近似调度器
比较 + 候选值
启动并行任务
网络 I/O 掩码
ECALL +
TEE 掩码辅助
门限值分享: left/right,
steep/flat
候选近似数据
- 比较结果
- base / 分段候选值
启动并行任务
网络 I/O 掩码
+可选Newton
TEE/本地选择
+ 可选Newton
本地 join / 同步
输出 z₁
结果重构
合并输出分享 -> 近似结果 / 误差检查

5. 数据流程图


1. 明文输入或测试向量
输入宽范围区间的明文 / 测试向量
2. 编码
将输入定点编码到 Z_L
3. 秘密分享
选择随机 x0
令 x1 = x - x0 mod L
4A. CSP0 本地设置
- 选择函数类型:
Sigmoid+/Tanh+/Erf+/Exp+/Inv+/Rsqrt+
- 加载 base 系数
- 加载分段阈值
- 准备 ECALL/网络缓冲区
4B. CSP1 本地设置
- 选择函数类型:
Sigmoid+/Tanh+/Erf+/Exp+/Inv+/Rsqrt+
- 加载 base 系数
- 加载分段阈值
- 准备 ECALL/网络缓冲区
5A. CSP0 并行在线阶段
网络路径
- 交换掩码分享
- 接收对方辅助材料
TEE / 本地辅助路径
- 准备阈值比较
- ODReLU辅助计算
- 输出 gate 分享
本地 join：等待两条路径完成
5B. CSP1 并行在线阶段
网络路径
- 交换掩码分享
- 接收对方辅助材料
TEE / 本地辅助路径
- 准备阈值比较
- ODReLU辅助计算
- 输出 gate 分享
本地 join：等待两条路径完成
6A. 宽范围候选近似
- Sigmoid+ /Tanh+ /Erf+ :
左侧饱和值常数
中间 base 近似
右侧饱和值常数
- EXP+ /Inv+ / InvSqrt+:
steep 候选值 flat 候选值
6B. 宽范围候选近似
- Sigmoid+ /Tanh+ /Erf+ :
左侧饱和值常数
中间 base 近似
右侧饱和值常数
- EXP+ /Inv+ / InvSqrt+:
steep 候选值 flat 候选值
7A. CSP0 并行在线阶段
网络路径
- gate 选择消息
- 可选Newton SMP/3SMP 消息
TEE / 本地辅助路径
- 饱和值合并
- gate 选择补偿
- 可选 Newton 修正
本地 join -> 输出z0
7B. CSP1 并行在线阶段
网络路径
- gate 选择消息
- 可选Newton SMP/3SMP 消息
TEE / 本地辅助路径
- 饱和值合并
- gate 选择补偿
- 可选 Newton 修正
本地 join -> 输出z1
8. 验证 / 统计
计算z = z0 + z1 mod L
将结果与明文数学函数进行比较

6. 运行方式

在 party0 和 party1 目录分别执行：

```
./1.sh
```

测试宏由 App/define/define.h 控制。输出包含各函数的输入范围、三种配置精度、通信量和逻辑轮次。